UD3 · El navegador como parte de la app

Módulo · Desarrollo Web en Entorno Cliente (DEW)

Apartado	Pág.
Qué vas a aprender	1
1. Los objetos predefinidos: qué tienes sin escribir nada	2
1.1. Tres familias de objetos	2
1.2. `window`: el objeto global	3
1.3. Propiedades, métodos y «objetos de utilidad»	3
1.4. ¿Por qué `'hola'.length` funciona si `'hola'` no es un objeto?	3
1.5. Explorar un objeto desde la consola	4
→ A1. Explorando los objetos predefinidos (CE a, h)	4
2. Cadenas y números a fondo	4
2.1. Métodos de `String` que usarás a diario	4
2.2. Quitar tildes y generar un *slug*	5
2.3. `Number`: precisión, conversión y comprobaciones	6
2.4. `Math`: redondeos, azar y lo demás	6
→ A2. Laboratorio de cadenas y números (CE a, h)	7
3. Fechas, importes e idiomas: `Date` e `Intl`	7
3.1. Crear fechas	7
3.2. Leer y modificar una fecha	8
3.3. Mostrar fechas y números: no lo hagas a mano	9
3.4. Otros formateadores útiles	9
→ A3. Fechas e importes bien formateados (CE a, c, h)	10
4. Hacer cosas más tarde: temporizadores	10
4.1. `setTimeout` y `setInterval`	10
4.2. Cancelar siempre lo que se repite	11
4.3. Lo que los temporizadores enseñan sobre el navegador	11
→ A4. Reloj y cuenta atrás (CE a, c, h)	12
5. Escribir en la página: generar texto y HTML desde código	12
5.1. De la consola a la página	12
5.2. `textContent`, `innerHTML` e `insertAdjacentHTML`	13
5.3. El peligro de `innerHTML`: inyección de código	13
5.4. `document.write`: por qué no	14
5.5. El título y otros detalles del documento	14
→ A5. La página habla (CE c, d, h)	14
6. Hablar con el navegador: el BOM	15
6.1. `location`: leer y cambiar la dirección	15
6.2. `history`: moverse por el historial	16
6.3. `navigator`: qué navegador, qué dispositivo, qué capacidades	16

Apartado	Pág.
6.4. `screen`, `window` y la apariencia de la ventana	16
→ A6. Radiografía del navegador (CE b, c, h)	17
7. Hablar con el usuario: diálogos	17
7.1. Los tres diálogos del navegador	17
7.2. Diálogos propios con `<dialog>`	18
7.3. Otras formas de avisar	19
→ A7. Diálogos propios (CE e, h)	19
8. Varias ventanas: abrir, cerrar y comunicar	19
8.1. Abrir ventanas y pestañas	19
8.2. Comunicar ventanas: `postMessage`	20
8.3. Comunicar pestañas del mismo origen: `BroadcastChannel` y el evento `storage`	20
→ A8. Ventanas que se hablan (CE b, f, h)	21
9. Recordar cosas: almacenamiento en el navegador	21
9.1. Cookies	22
9.2. `localStorage` y `sessionStorage`	23
9.3. El evento `storage`	24
9.4. IndexedDB: cuando `localStorage` se queda pequeño	24
9.5. Verlo todo: la pestaña Application	25
→ A9. Guardar y recuperar (CE g, h)	25
10. Privacidad: lo que la ley y el sentido común exigen	26
10.1. Qué es un dato personal y qué tiene que ver contigo	26
10.2. Buenas prácticas en el código	26
10.3. El aviso de consentimiento del proyecto	26
11. Depurar y documentar lo que no se ve	27
11.1. Depurar temporizadores y almacenamiento	27
11.2. Documentar objetos de datos con JSDoc	27
→ A10. Preferencias de tu proyecto (producto de la unidad; CE a–h)	27
12. Resumen de actividades y entregas	28
13. Autoevaluación	29
14. Recursos	29

Resultado de aprendizaje	RA3. Escribe código, identificando y aplicando las funcionalidades aportadas por los objetos predefinidos del lenguaje.
Duración	8 horas (semanas 4 y 5)
Hito de la unidad	Tus programas dejan de vivir solo en la consola: formatean fechas e importes como lo haría una aplicación real, escriben en la página, hablan con el navegador (URL, ventanas, pantalla) y recuerdan las preferencias del usuario entre visitas. Tu proyecto estrena su página de preferencias (R6).

Qué vas a aprender

Al terminar esta unidad serás capaz de:

- Distinguir los tres grupos de objetos que tienes disponibles sin escribir nada: los **nativos del lenguaje** (String, Number, Math, Date, JSON…), los del **navegador** (window, location, navigator, localStorage…) y los del **documento** (document, que verás a fondo en la UD6).
- Manipular cadenas y números con sus métodos, y formatear fechas, importes y porcentajes en el idioma y la moneda del usuario con Intl.
- Programar tareas que ocurren «más tarde» o «cada cierto tiempo» con setTimeout y setInterval, y cancelarlas.
- Generar texto y HTML desde código sin abrir agujeros de seguridad.
- Leer y modificar la URL, el historial, el título y la apariencia de la ventana, y consultar las capacidades del navegador.
- Pedir confirmaciones y datos al usuario con diálogos del navegador y con diálogos propios (<dialog>).
- Abrir ventanas nuevas y comunicar ventanas y pestañas entre sí.
- Guardar y recuperar información en el navegador con cookies, localStorage y sessionStorage, saber qué guarda cada mecanismo y cuánto dura, y conocer para qué sirve IndexedDB.
- Respetar la privacidad del usuario: qué exige el RGPD, qué necesita consentimiento y cómo pedirlo.
- Depurar el almacenamiento y los temporizadores con la pestaña **Application** de las DevTools y documentar tu código.

Criterios de evaluación (RA3)

CE	Se evaluará si…
a	Has identificado los objetos predefinidos del lenguaje.
b	Has analizado los objetos referentes a las ventanas del navegador y los documentos web que contienen.
c	Has escrito sentencias que utilizan los objetos predefinidos del lenguaje para cambiar el aspecto del navegador y el documento que contiene.
d	Has generado textos y etiquetas como resultado de la ejecución de código en el navegador.
e	Has escrito sentencias que utilizan los objetos predefinidos del lenguaje para interactuar con el usuario.
f	Has utilizado las características propias del lenguaje en documentos compuestos por varias ventanas.
g	Has utilizado mecanismos del navegador web para almacenar información y recuperar su contenido.
h	Has depurado y documentado el código.

Cómo está organizada esta unidad

En esta unidad las actividades van **intercaladas con la teoría**: cada apartado termina con la actividad que lo pone en práctica, para que la hagas en cuanto termines de leerlo. Hay diez actividades (A1 a A10); la **A10** es el producto evaluable y se construye con piezas de todas las anteriores. En el apartado 12 tienes el resumen de todas las entregas.

Todas las actividades se hacen en tu repositorio dwec-ejercicios, dentro de la carpeta ud03, una subcarpeta por actividad (a1-objetos, a2-cadenas…). La A10 va en dwec-proyecto. Haz un commit por actividad terminada y push al acabar cada sesión. Salvo que se indique lo contrario, las actividades son individuales.

A diferencia de la UD2, en esta unidad el index.html **ya no está vacío**: tus programas escribirán en la página. Este es el punto de partida de cada actividad (cambia el título y el h1):

```
<!DOCTYPE html>
<html lang="es">
<head>
 <meta charset="UTF-8">
 <meta name="viewport" content="width=device-width, initial-scale=1">
 <title>UD3 · A1</title>
 <link rel="stylesheet" href="estilos.css">
 <script src="app.js" defer></script>
</head>
<body>
 <h1>UD3 · A1 · Objetos predefinidos</h1>
 <main id="salida"></main>
</body>
</html>
```

El elemento <main id="salida"> es el «hueco» en el que escribirán tus programas (apartado 5). El fichero estilos.css es opcional: lo que importa en este módulo es el JavaScript, pero una hoja de estilos mínima hace más agradable ver el resultado.

> *Importante. Abre siempre las páginas con Live Server (http://127.0.0.1:5500/...), nunca haciendo doble clic en el fichero (file://). Varias cosas de esta unidad (cookies, localStorage compartido entre páginas, postMessage, BroadcastChannel) dependen del origen de la página, y con file:// no funcionan o funcionan de forma distinta. Y usa siempre la misma dirección: 127.0.0.1:5500 y localhost:5500 son orígenes distintos y no comparten almacenamiento.*

1. Los objetos predefinidos: qué tienes sin escribir nada

1.1. Tres familias de objetos

En la UD2 usaste console.log, prompt, Number() o 'texto'.toUpperCase() sin preguntarte de dónde salían. Todos son objetos predefinidos: están disponibles en cuanto arranca tu programa, sin que tengas que crearlos ni importarlos. Conviene saber a qué familia pertenece cada uno, porque eso te dice dónde existe y dónde buscar su documentación.

Familia	Quién la define	Ejemplos	¿Existe en Node.js?
Objetos nativos del lenguaje	El estándar ECMAScript	Object, String, Number, Boolean, Math, Date, JSON, RegExp, Array, Map, Set, Promise, Intl, Error	Sí: son el lenguaje
Objetos del navegador (BOM, *Browser Object Model*)	Los estándares web (WHATWG, W3C)	window, location, history, navigator, screen, localStorage, sessionStorage, console, alert, setTimeout, fetch	Algunos sí (console, setTimeout, fetch), otros no (window, alert, localStorage)
Objetos del documento (DOM, *Document Object Model*)	Los estándares web	document y todo lo que cuelga de él: elementos, atributos, texto	No: no hay documento

La primera familia es el lenguaje; las otras dos son el entorno de ejecución. Por eso en la UD2 alert('hola') fallaba en Node.js: no es que el lenguaje no lo tenga, es que ese entorno no lo tiene. Esta unidad se dedica a las dos primeras familias (y a un anticipo de la tercera); la UD6 se dedica entera al DOM.

> *Consejo. La referencia de todos estos objetos está en MDN. Para los nativos, busca «MDN String» o «MDN Date»; para los del navegador, «MDN Window» o «MDN localStorage». Acostúmbrate a leer la tabla de compatibilidad del final de cada página: te dice desde qué versión de cada navegador existe cada cosa.*

1.2. window: el objeto global

En el navegador, el objeto window representa la ventana (o pestaña) y además es el objeto global: todo lo que no cuelga de nada cuelga de él. alert(…) y window.alert(…) son lo mismo; location y window.location también. Por eso normalmente se omite.

```
console.log(window.innerWidth); // ancho de la zona visible, en píxeles
console.log(innerWidth);        // lo mismo
console.log(typeof window.alert, typeof window.document, typeof window.location);
// 'function' 'object' 'object'
```

Una consecuencia que ya conoces de la UD2: una variable declarada con var fuera de cualquier función se convierte en propiedad de window (var x = 1; window.x // 1); con let y const no ocurre. Es una razón más para no usar var.

1.3. Propiedades, métodos y «objetos de utilidad»

Un objeto agrupa propiedades (datos) y métodos (funciones). Algunos objetos predefinidos son objetos de utilidad: no se crean instancias de ellos, solo se usan sus métodos y constantes. Math, JSON e Intl son así:

```
Math.PI;          // 3.141592653589793  (propiedad)
Math.round(2.5);  // 3                  (método)
JSON.stringify({ a: 1 }); // '{"a":1}'  (método)
```

Otros son constructores: con new crean un valor nuevo de ese tipo. Date es el ejemplo típico de esta unidad:

```
const ahora = new Date();     // un objeto fecha con el instante actual
ahora.getFullYear();          // método de la instancia
Date.now();                   // método «estático»: se llama sobre Date, no sobre una fecha
```

Fíjate en la diferencia entre un método de instancia (ahora.getFullYear(), actúa sobre una fecha concreta) y un método estático (Date.now(), pertenece al constructor). En MDN los verás separados en «Static methods» e «Instance methods».

1.4. ¿Por qué 'hola'.length funciona si 'hola' no es un objeto?

Los primitivos (string, number, boolean) no son objetos, pero cuando accedes a una propiedad o llamas a un método sobre uno de ellos, JavaScript lo envuelve temporalmente en el objeto correspondiente (String, Number, Boolean), ejecuta el método y descarta el envoltorio. A esto se le llama *autoboxing*. Es lo que hace posible escribir 'hola'.toUpperCase() o (3.14159).toFixed(2). No necesitas (ni debes) escribir new String('hola'): crearías un objeto de verdad, con comportamientos raros en las comparaciones.

```
typeof 'hola';            // 'string'
typeof new String('hola'); // 'object'  ← no lo hagas
'hola' === 'hola';        // true
new String('hola') === new String('hola'); // false (son dos objetos distintos)
```

1.5. Explorar un objeto desde la consola

La consola es la mejor herramienta para descubrir qué ofrece un objeto:

- Escribe el nombre y un punto (navigator.) y la consola te autocompleta las propiedades y métodos.
- console.dir(objeto) lo muestra como árbol desplegable, incluidas las propiedades heredadas.
- Object.keys(objeto) devuelve los nombres de sus propiedades propias en un array (los arrays los verás en la UD4, pero ya puedes leer el resultado).
- typeof objeto.propiedad te dice si es un dato o un método ('function').

```
console.dir(location);
Object.keys(localStorage);   // las claves guardadas (de momento, ninguna)
typeof navigator.language;   // 'string'
typeof navigator.share;      // 'function' (o 'undefined' si el navegador no lo soporta)
```

Esa última línea es un patrón importante: detectar si una capacidad existe antes de usarla (if (typeof navigator.share === 'function') { … } o, más corto, if (navigator.share) { … }). Lo aplicarás varias veces en esta unidad.

A1. Explorando los objetos predefinidos (CE a, h)

Objetivo: identificar los objetos predefinidos disponibles, clasificarlos por familia y comprobar en qué entornos existen.

1. En la consola de las DevTools, para cada uno de estos nombres, anota su typeof y a qué familia pertenece (nativo del lenguaje, navegador o documento): Math, Date, JSON, window, document, location, navigator, localStorage, console, setTimeout, alert, Intl, screen, history.
2. Crea ut03/a1-objetos con el index.html de partida y un app.js que muestre por consola, con console.table, una tabla con al menos ocho de esos nombres y dos columnas: tipo (el resultado de typeof) y familia. Pista: de momento puedes construir la tabla con un objeto literal por fila, como hiciste con console.table en la UD2.
3. Ejecuta el mismo app.js con Node.js. Anota cuáles de esos nombres dan ReferenceError y explica por qué con la tabla del apartado 1.1. Modifica el programa para que **no falle** en Node: usa typeof nombre (que nunca lanza error) en lugar de acceder al nombre directamente.

4. Con console.dir(navigator) y el autocompletado, localiza tres propiedades de navigator que no aparezcan en estos apuntes y anota qué crees que indican. Comprueba tu hipótesis en MDN.
5. Averigua cuántas propiedades y métodos tiene Math: prueba primero Object.keys(Math).length y después Object.getOwnPropertyNames(Math).length, y anota por qué no coinciden (pista: busca «enumerable» en MDN). Con el autocompletado localiza el método que calcula el máximo de varios números y el que devuelve el valor absoluto. Comprueba Math.max(3, 9, 4) y Math.abs(-7).

Entrega: la carpeta a1-objetos y un NOTAS.md con la clasificación del punto 1, los resultados del punto 3 y las respuestas del punto 4.

2. Cadenas y números a fondo

En la UD2 usaste algunos métodos de cadenas y de números sin detenerte en ellos. Ahora los vas a necesitar de verdad: para formatear, limpiar entradas del usuario, generar identificadores o textos y preparar lo que mostrarás en la página.

2.1. Métodos de String que usarás a diario

Recuerda que las cadenas son inmutables: ningún método modifica la cadena original, todos devuelven una nueva. Si quieres conservar el resultado, guárdalo.

Método	Qué hace	Ejemplo	Resultado
length (propiedad)	Número de caracteres	'Tenerife'.length	8
at(i)	Carácter en la posición i; admite negativos	'Tenerife'.at(-1)	'e'
toUpperCase(), toLowerCase()	Mayúsculas / minúsculas	'Daw'.toUpperCase()	'DAW'
trim(), trimStart(), trimEnd()	Quita espacios de los extremos	' hola '.trim()	'hola'
includes(s)	¿Contiene s?	'Canarias'.includes('nar')	true
startsWith(s), endsWith(s)	¿Empieza / termina por s?	'fichero.js'.endsWith('.js')	true
indexOf(s), lastIndexOf(s)	Posición de s (o -1)	'banana'.indexOf('na')	2
slice(ini, fin)	Subcadena desde ini hasta fin (sin incluir); admite negativos	'JavaScript'.slice(0, 4)	'Java'
substring(ini, fin)	Como slice, pero sin negativos	'JavaScript'.substring(4)	'Script'
replace(a, b), replaceAll(a, b)	Sustituye la primera / todas las apariciones	'a-b-c'.replaceAll('-', '/')	'a/b/c'
split(sep)	Parte la cadena en un array	'1,2,3'.split(',')	['1', '2', '3']
repeat(n)	Repite la cadena	'ab'.repeat(3)	'ababab'
padStart(n, c), padEnd(n, c)	Rellena hasta n caracteres	'7'.padStart(2, '0')	'07'
localeCompare(s)	Compara según el idioma (para ordenar)	'ñ'.localeCompare('o', 'es')	-1
normalize('NFD')	Descompone los caracteres acentuados	ver apartado 2.2	

```
const nombre = '  ada LOVELACE ';
const limpio = nombre.trim();                  // 'ada LOVELACE'
const partes = limpio.split(' ');              // ['ada', 'LOVELACE']
const inicial = partes[0].at(0).toUpperCase();     // 'A'
```

```
const resto = partes[0].slice(1).toLowerCase();    // 'da'
console.log(`${inicial}${resto} ${partes[1].toUpperCase()}`); // 'Ada LOVELACE'
```

La notación partes[0] accede al primer elemento del array que devuelve split. Los arrays son el tema central de la UD4; aquí solo necesitas saber que se indexan desde cero.

> *Consejo. slice admite índices negativos ('2026-10-13'.slice(-2) es '13'), lo que lo hace más cómodo que substring. Elige uno y úsalo siempre: en este módulo, slice.*

2.2. Quitar tildes y generar un *slug*

Un caso práctico que combina varios métodos: convertir un título en un identificador apto para una URL o para un nombre de fichero (lo que se llama *slug*). normalize('NFD') separa cada letra acentuada en la letra base más un carácter de acento, y después se eliminan esos caracteres de acento, que ocupan el rango Unicode `\u0300`–`\u036f`. Para eliminarlos usamos replace con una expresión regular, un patrón de texto escrito entre barras. Las expresiones regulares se estudian en la UD7; aquí solo necesitas copiar este patrón:

```
const titulo = 'Canción de Navidad (edición 2026)';
const slug = titulo
  .normalize('NFD')              // 'Cancio' + '´' + 'n de Navidad …'
  .replace(/[\u0300-\u036f]/g, '')   // quita los acentos: 'Cancion de Navidad (edicion 2026)'
  .toLowerCase()
  .trim()
  .replace(/[^a-z0-9]+/g, '-')       // todo lo que no sea letra o dígito → guion
  .replace(/^-+|-+$/g, '');          // quita guiones de los extremos
console.log(slug); // 'cancion-de-navidad-edicion-2026'
```

Fíjate en el encadenamiento: como cada método devuelve una cadena nueva, puedes llamar al siguiente directamente sobre el resultado. Escribir cada llamada en su línea, como arriba, hace el encadenamiento legible.

2.3. Number: precisión, conversión y comprobaciones

Qué	Cómo	Ejemplo
Convertir texto a número	Number(texto), parseInt(texto, 10), parseFloat(texto)	Number('3.5') → 3.5; parseInt('42px', 10) → 42
Redondear a n decimales (devuelve texto)	n.toFixed(d)	(2.345).toFixed(2) → '2.35' (ojo: (2.5).toFixed(0) → '3', pero (1.005).toFixed(2) → '1.00')
¿Es entero? ¿Es finito? ¿Es NaN?	Number.isInteger(x), Number.isFinite(x), Number.isNaN(x)	Number.isInteger(3.0) → true
Límites	Number.MAX_SAFE_INTEGER, Number.EPSILON	2 ** 53 - 1
Notación científica	n.toExponential(d)	(123456).toExponential(2) → '1.23e+5'

Dos avisos que te ahorrarán horas:

- toFixed devuelve una cadena. Si vas a seguir calculando, no la uses; úsala solo para mostrar (y en el apartado 3 verás que Intl lo hace mejor).
- Para redondear a dos decimales como número, usa Math.round(x * 100) / 100. Y recuerda de la UD2 que 0.1 + 0.2 no es exactamente 0.3: si comparas importes, compara céntimos enteros.

2.4. Math: redondeos, azar y lo demás

Math es un objeto de utilidad: todas sus propiedades y métodos son estáticos.

Método	Qué hace	Ejemplo
Math.round(x)	Redondea al entero más próximo (el .5 sube)	Math.round(2.5) → 3; Math.round(-2.5) → -2
Math.floor(x), Math.ceil(x)	Redondea hacia abajo / hacia arriba	Math.floor(2.9) → 2; Math.ceil(2.1) → 3
Math.trunc(x)	Quita los decimales (sin redondear)	Math.trunc(-2.9) → -2
Math.abs(x)	Valor absoluto	Math.abs(-7) → 7
Math.max(…), Math.min(…)	Máximo y mínimo de varios valores	Math.max(3, 9, 4) → 9
Math.pow(b, e), Math.sqrt(x), Math.cbrt(x)	Potencia (igual que **), raíz cuadrada, raíz cúbica	Math.sqrt(144) → 12
Math.random()	Número aleatorio en [0, 1)	0.7310585…
Math.PI, Math.E	Constantes	3.14159…

El uso más frecuente de Math.random() es obtener un entero aleatorio entre dos valores, ambos incluidos. Memoriza este patrón:

```
// Entero aleatorio entre min y max, ambos incluidos
const min = 1;
const max = 6;
const dado = Math.floor(Math.random() * (max - min + 1)) + min;
```

Math.random() sirve para juegos, ejemplos y datos de prueba, pero **no** para seguridad (contraseñas, tokens): para eso existe crypto.randomUUID(), que genera un identificador único, y crypto.getRandomValues(). Verás crypto.randomUUID() en la UD4 para dar identificadores a los elementos de tu colección.

A2. Laboratorio de cadenas y números (CE a, h)

Objetivo: aplicar los métodos de String, Number y Math para limpiar, transformar y generar valores.

Crea ut03/a2-cadenas con el index.html de partida y un fichero por punto (1-nombres.js, 2-slug.js…), todos enlazados con <script defer>. Cada programa pide sus datos con prompt (todavía) y muestra el resultado por consola. Documenta con JSDoc las funciones que escribas.

1. Nombre normalizado. Pide un nombre y apellidos escritos de cualquier manera (' ada LOVELACE byron ') y muestra: el nombre con cada palabra en mayúscula inicial y el resto en minúscula, las iniciales ('A. L. B.') y el número de palabras. Pista: trim, split(' '), at(0), slice(1), un bucle for…of.
2. Slug y nombre de fichero. Pide un título y muestra su *slug* (apartado 2.2) y un nombre de fichero con la fecha de hoy delante, con el formato 2026-10-13-titulo-del-articulo.md. Para la fecha usa de momento new Date().toISOString().slice(0, 10) (lo entenderás en el apartado 3).
3. Censura. Pide una frase y una palabra, y muestra la frase con todas las apariciones de esa palabra (sin distinguir mayúsculas) sustituidas por asteriscos de la misma longitud ('*'.repeat(palabra.length)). Pista: compara en minúsculas, pero conserva el original; replaceAll solo acepta texto literal, así que si no distingues mayúsculas tendrás que recorrer con indexOf en bucle o convertir todo a minúsculas antes.
4. Dados y porcentajes. Escribe una función enteroAleatorio(min, max) con el patrón del apartado 2.4 y úsala para simular 1000 tiradas de un dado de seis caras, contando cuántas veces sale cada cara (seis contadores) y mostrando el porcentaje de cada una con dos decimales. ¿Se acercan al 16,67 %?
5. Contraseña provisional. Genera una contraseña de 10 caracteres eligiendo al azar letras y dígitos de la cadena 'ABCDEFGHJKLMNPQRSTUVWXYZabcdefghijkmnpqrstuvwxyz23456789' (se han quitado los caracteres que se confunden). Muestra también crypto.randomUUID() y explica en un comentario por qué el segundo es adecuado para identificadores y tu función no lo sería para contraseñas reales.

6. Redondeos. Muestra en una tabla (console.table) el resultado de round, floor, ceil, trunc y toFixed(0) para los valores 2.5, -2.5, 2.4999 y 1.005. Comenta en el código las dos filas que más te sorprendan.

Entrega: la carpeta a2-cadenas y PRUEBAS.md con una tabla de al menos tres casos de prueba (entrada · esperado · obtenido) para los puntos 1, 2 y 3.

3. Fechas, importes e idiomas: Date e Intl

3.1. Crear fechas

Date representa un instante con precisión de milisegundos. Internamente guarda el número de milisegundos transcurridos desde el 1 de enero de 1970 a las 00:00 UTC (la marca de tiempo o *timestamp*).

```
const ahora = new Date();                  // instante actual
const fijo = new Date(2026, 9, 13, 8, 30);     // 13 de octubre de 2026, 8:30 (¡mes 9 = octubre!)
const iso = new Date('2026-10-13T08:30:00');    // desde texto ISO 8601, hora local
const soloFecha = new Date('2026-10-13');      // ¡cuidado! esto se interpreta como UTC
const desdeMs = new Date(0);               // 1 de enero de 1970, 00:00 UTC
Date.now();                          // milisegundos actuales, como número
```

Tres trampas clásicas:

- Los meses empiezan en 0: enero es 0 y diciembre 11. Los días del mes, en cambio, empiezan en 1. Es la fuente de errores más habitual con fechas.
- Una cadena solo con fecha ('2026-10-13') se interpreta como medianoche UTC, mientras que una cadena con fecha y hora sin zona ('2026-10-13T08:30:00') se interpreta en hora local. En Canarias (UTC+0 en invierno, UTC+1 en verano) la diferencia puede cambiar el día.
- Si pasas una cadena en otro formato ('13/10/2026'), el resultado depende del navegador. Usa siempre ISO 8601 (AAAA-MM-DD) para intercambiar fechas como texto: es lo que devuelve toISOString() y lo que usan los <input type="date">.

3.2. Leer y modificar una fecha

Leer	Modificar	Significado
getFullYear()	setFullYear(a)	Año con cuatro cifras
getMonth()	setMonth(m)	Mes, de 0 a 11
getDate()	setDate(d)	Día del mes, de 1 a 31
getDay()	—	Día de la semana, de 0 (domingo) a 6 (sábado)
getHours(), getMinutes(), getSeconds(), getMilliseconds()	setHours(h, m, s, ms)…	Hora local
getTime()	setTime(ms)	Milisegundos desde 1970
getTimezoneOffset()	—	Diferencia con UTC en minutos

Las versiones getUTCHours(), getUTCDate()… leen en UTC en lugar de en hora local.

Los setters desbordan de forma inteligente, y eso permite hacer aritmética de fechas sin calcular a mano cuántos días tiene cada mes:

```
const f = new Date(2026, 0, 31);  // 31 de enero
f.setDate(f.getDate() + 1);      // 1 de febrero: el desbordamiento pasa de mes solo
f.setMonth(f.getMonth() + 12);   // un año después, 1 de febrero de 2027
```

```
// Diferencia en días entre dos fechas: restar da milisegundos
const inicio = new Date(2026, 8, 14);
const fin = new Date(2026, 11, 18);
const dias = Math.round((fin - inicio) / (1000 * 60 * 60 * 24)); // 95
```

Al restar dos fechas, JavaScript las convierte a milisegundos (es una conversión implícita, como las de la UD2). Math.round evita que un cambio de hora de verano a invierno te deje un 94.958….

> *Nota. Date tiene un diseño antiguo y el estándar prepara un sustituto, Temporal, con fechas inmutables, zonas horarias explícitas y meses que empiezan en 1. Ya está disponible en los navegadores más recientes y acabará siendo lo habitual, pero durante este curso trabajamos con Date, que es lo que encontrarás en la inmensa mayoría del código existente y en las librerías.*

3.3. Mostrar fechas y números: no lo hagas a mano

Podrías montar una fecha con getDate() + '/' + (getMonth() + 1)… y un importe con toFixed(2) + ' €'. No lo hagas: te saldrán 5/3/2026 donde un usuario alemán espera 05.03.2026 y 1234.50 € donde un español espera 1.234,50 €. El objeto Intl (internacionalización) formatea fechas, números, monedas, listas y tiempos relativos según el idioma y las convenciones de cada región, y lo hace mejor que cualquier función que escribas tú.

Un formateador se crea una vez (con el idioma y las opciones) y se usa muchas veces con format():

```
const fecha = new Date(2026, 9, 13, 8, 30);

new Intl.DateTimeFormat('es-ES').format(fecha);              // '13/10/2026'
new Intl.DateTimeFormat('en-US').format(fecha);              // '10/13/2026'
new Intl.DateTimeFormat('es-ES', { dateStyle: 'long' }).format(fecha);
// '13 de octubre de 2026'
new Intl.DateTimeFormat('es-ES', { dateStyle: 'full', timeStyle: 'short' }).format(fecha);
// 'martes, 13 de octubre de 2026, 8:30'
new Intl.DateTimeFormat('es-ES', { weekday: 'long', day: 'numeric', month: 'short' }).format(fecha);
// 'martes, 13 oct'
new Intl.DateTimeFormat('es-ES', { hour: '2-digit', minute: '2-digit', second: '2-digit' }).format(fecha);
// '08:30:00'
```

Los métodos fecha.toLocaleDateString('es-ES', opciones) y fecha.toLocaleTimeString(…) son atajos que crean el formateador por dentro; son cómodos para una sola fecha, pero si vas a formatear muchas (una tabla, un listado) crea el formateador una vez y reutilízalo.

```
const precio = 1234.5;

new Intl.NumberFormat('es-ES').format(precio);               // '1234,5'
new Intl.NumberFormat('es-ES', { style: 'currency', currency: 'EUR' }).format(precio);
// '1234,50 €'
new Intl.NumberFormat('en-US', { style: 'currency', currency: 'USD' }).format(precio);
// '$1,234.50'
new Intl.NumberFormat('de-DE', { style: 'currency', currency: 'EUR' }).format(precio);
// '1.234,50 €'
new Intl.NumberFormat('es-ES', { style: 'percent', maximumFractionDigits: 1 }).format(0.1667);
// '16,7 %'
new Intl.NumberFormat('es-ES', { style: 'unit', unit: 'kilometer' }).format(42.195);
// '42,195 km'
```

```
new Intl.NumberFormat('es-ES', { minimumFractionDigits: 2, maximumFractionDigits: 2 }).format(7);
// '7,00'
```

Fíjate en que en español el separador de miles solo aparece a partir de cinco cifras (1234,50 € pero 12.345,50 €): es la norma ortográfica, y Intl la conoce. También sabe que el símbolo del euro va detrás en español y delante en inglés, y que el porcentaje lleva un espacio en español.

3.4. Otros formateadores útiles

```
// Tiempo relativo: «hace 3 días», «dentro de 2 horas»
const rtf = new Intl.RelativeTimeFormat('es', { numeric: 'auto' });
rtf.format(-3, 'day');  // 'hace 3 días'
rtf.format(1, 'day');   // 'mañana'  (gracias a numeric: 'auto')
rtf.format(2, 'hour');  // 'dentro de 2 horas'

// Listas: «a, b y c»
new Intl.ListFormat('es', { type: 'conjunction' }).format(['HTML', 'CSS', 'JavaScript']);
// 'HTML, CSS y JavaScript'

// Plurales: elegir la forma correcta
const pr = new Intl.PluralRules('es');
pr.select(1); // 'one'
pr.select(3); // 'other'
```

Y el dato que une todo esto con el usuario real: navigator.language devuelve el idioma preferido del navegador ('es-ES', 'en-GB'…). Si pasas navigator.language como primer argumento a cualquier formateador, el resultado se adapta a cada usuario sin que tengas que hacer nada más. En el proyecto, el idioma será además una de las preferencias que el usuario podrá elegir y que guardarás en el navegador (apartado 9).

> ***Consejo.*** *Si no indicas idioma (new Intl.NumberFormat()), se usa el del navegador. En clase, con todos los equipos en español, no notarás la diferencia; en la vida real sí. Pasa siempre el idioma de forma explícita: el del usuario, el de la preferencia guardada o 'es-ES' como valor por defecto.*

A3. Fechas e importes bien formateados (CE a, c, h)

Objetivo: operar con fechas y presentar fechas, números e importes localizados con Intl.

Crea ut03/a3-fechas con el index.html de partida y un fichero por punto.

1. Edad exacta. Pide la fecha de nacimiento en formato ISO (AAAA-MM-DD) y muestra la edad en años cumplidos (cuidado con quienes aún no han cumplido años este año), el día de la semana en que nació (con Intl, en español) y cuántos días faltan para el próximo cumpleaños. Valida que la fecha sea correcta: new Date(texto) con un texto inválido produce una fecha cuyo getTime() es NaN.
2. Cuenta de días. Muestra cuántos días faltan para el final de la primera evaluación (18 de diciembre de 2026) y para el final de las clases en el centro (5 de febrero de 2027), y exprésalo también con Intl.RelativeTimeFormat («dentro de 68 días»). Si la fecha ya pasó, debe decir «hace N días».
3. Mismo importe, cuatro países. Pide un importe y muestra, con console.table, cómo lo vería un usuario en es-ES (EUR), en-US (USD), de-DE (EUR), ja-JP (JPY) y en-IN (INR). Observa la agrupación de cifras en en-IN y los decimales en ja-JP.
4. Tabla de precios localizada. Con cinco precios fijos en una cadena separada por comas ('12.5,8,1250,0.99,19999'), muestra una tabla con cada precio formateado en euros, con IGIC del 7 % aplicado (también formateado) y el porcentaje de impuesto formateado con style: 'percent'. Crea un solo formateador de moneda y reutilízalo.

5. El idioma del usuario. Muestra navigator.language y navigator.languages, y vuelve a formatear la fecha de hoy con dateStyle: 'full' usando ese idioma. Cambia el idioma preferido del navegador en su configuración (o usa otro navegador), recarga y anota qué cambia.

Entrega: la carpeta a3-fechas y PRUEBAS.md con tres casos de prueba para el punto 1 (incluye uno de alguien que cumple años hoy y uno con una fecha inválida).

4. Hacer cosas más tarde: temporizadores

4.1. setTimeout y setInterval

Hasta ahora tus programas se ejecutaban de arriba abajo y terminaban. El navegador también puede ejecutar código después de un tiempo o cada cierto tiempo:

```
// Ejecuta la función UNA vez, pasados 2000 ms
const idAviso = setTimeout(() => {
  console.log('Han pasado dos segundos');
}, 2000);

// Ejecuta la función CADA 1000 ms, hasta que se cancele
let segundos = 0;
const idReloj = setInterval(() => {
  segundos += 1;
  console.log(`${segundos} s`);
}, 1000);

// Cancelar: con el identificador que devolvieron
clearTimeout(idAviso);
clearInterval(idReloj);
```

Lo que pasas como primer argumento es una función: no la llamas (no lleva paréntesis al final), la entregas para que el navegador la llame más tarde. A una función que se entrega para que otro la llame se le denomina callback (*función de retrollamada*). Aquí está escrita como función flecha (() => { … }), una forma breve de escribir funciones que estudiarás en la UD4; de momento, léela como «una función sin nombre que hace esto». También puedes pasar el nombre de una función ya declarada:

```
function saludar() {
  console.log('Hola');
}
setTimeout(saludar, 1000);  // correcto: entregas la función
setTimeout(saludar(), 1000); // ERROR típico: la llamas ahora y entregas undefined
```

Ese segundo caso es el error más habitual con temporizadores: los paréntesis ejecutan la función inmediatamente y setTimeout recibe lo que devuelve (nada). Si necesitas pasarle argumentos, envuélvela: setTimeout(() => saludar('Ada'), 1000).

4.2. Cancelar siempre lo que se repite

Un setInterval que no se cancela sigue ejecutándose mientras la página esté abierta, aunque el elemento que actualizaba ya no exista. Guarda siempre el identificador y cancélalo cuando termine la tarea. Este es el patrón de una cuenta atrás:

```
let restantes = 10;
const id = setInterval(() => {
```

```
  restantes -= 1;
  console.log(restantes);
  if (restantes === 0) {
    clearInterval(id);      // se cancela a sí misma
    console.log('¡Tiempo!');
  }
}, 1000);
```

Para tareas que deben repetirse pero cuya duración puede variar (una petición a un servidor, por ejemplo), es más seguro un setTimeout que se vuelve a programar al terminar, en lugar de un setInterval que podría solaparse.

4.3. Lo que los temporizadores enseñan sobre el navegador

Prueba esto en la consola y predice el orden de salida antes de ejecutarlo:

```
console.log('1');
setTimeout(() => console.log('2'), 0);
console.log('3');
// Salida: 1, 3, 2
```

Aunque el retardo sea 0, el '2' sale el último. El navegador ejecuta tu programa de principio a fin sin interrupciones y solo cuando ha terminado atiende la cola de tareas pendientes (temporizadores vencidos, clics del usuario, respuestas del servidor). Es el bucle de eventos (*event loop*), que estudiarás en la UD8 con las promesas. Dos consecuencias prácticas:

- El retardo de setTimeout es un mínimo, no una garantía: si el programa está ocupado, la función se retrasa.
- Un bucle while (true) o un cálculo muy largo congela la página: ni temporizadores, ni clics, ni repintado. Por eso en la UD2 un bucle infinito bloqueaba el navegador.

Para animaciones, el navegador ofrece algo mejor que setInterval(…, 16): **requestAnimationFrame(callback)** ejecuta la función justo antes del siguiente repintado (unas 60 veces por segundo) y se detiene cuando la pestaña no está visible. Lo usarás en la UD6.

A4. Reloj y cuenta atrás (CE a, c, h)

Objetivo: programar tareas periódicas y diferidas, cancelarlas y combinarlas con Intl.

Crea ut03/a4-tiempo con el index.html de partida. En esta actividad el resultado se muestra **en la página**: de momento, usa document.getElementById('salida').textContent = texto; para escribir texto dentro del <main> (lo explicamos en el apartado 5; aquí basta con copiarlo).

1. Reloj. Muestra la hora actual en la página con formato HH:MM:SS (Intl.DateTimeFormat con hour, minute y second a '2-digit') y actualízala cada segundo. Añade debajo la fecha completa (dateStyle: 'full') en el idioma del navegador. Comprueba con la pestaña Performance o simplemente observando que el reloj no «salta» dos segundos de golpe.
2. Cuenta atrás con cancelación. Pide con prompt un número de segundos y haz una cuenta atrás en la página; al llegar a cero muestra «¡Tiempo!» y cancela el intervalo. Comprueba con un console.log dentro de la función que, tras el cero, deja de ejecutarse.
3. El orden de las cosas. Escribe un programa con cinco console.log y tres setTimeout con retardos 0, 100 y 50, mezclados en cualquier orden. Escribe en un comentario, antes de ejecutarlo, el orden de salida que predices; ejecútalo y anota el real.
4. Trampa. Escribe setTimeout(saludar(), 1000) con una función saludar que escribe en consola. Anota qué ocurre, por qué, y corrígelo de las dos formas posibles (sin paréntesis y con función flecha envolvente).

5. Ampliación · Pomodoro. Un temporizador de 25 minutos de trabajo y 5 de descanso que alterna automáticamente, muestra el tiempo restante como MM:SS y cambia el document.title para que se vea el tiempo en la pestaña aunque no esté activa. Para probarlo, parametriza las duraciones en segundos.

Entrega: la carpeta a4-tiempo con un fichero por punto y las predicciones del punto 3 como comentario en el código.

5. Escribir en la página: generar texto y HTML desde código

5.1. De la consola a la página

La consola es para ti; la página es para el usuario. Para escribir en ella necesitas dos cosas: **localizar** un elemento del documento y **cambiar su contenido**. El objeto document (la tercera familia del apartado 1) te da ambas:

```
const salida = document.getElementById('salida'); // el <main id="salida"> del index.html
salida.textContent = 'Hola desde JavaScript';   // sustituye su contenido por texto
```

getElementById devuelve el elemento con ese id (o null si no existe: comprueba el id y que el script tenga defer, porque sin defer el script se ejecuta antes de que exista el <main>). Guardas la referencia en una constante y la reutilizas. Esto es el DOM, y la UD6 se dedica entera a él; en esta unidad solo usarás getElementById y las tres formas de escribir que vienen a continuación.

5.2. textContent, innerHTML e insertAdjacentHTML

Propiedad / método	Qué hace	Cuándo usarlo
el.textContent = texto	Sustituye el contenido por texto plano; las etiquetas se ven como texto	Siempre que lo que escribes es texto, sobre todo si viene del usuario
el.innerHTML = html	Sustituye el contenido interpretando el texto como HTML	Cuando generas marcado tú mismo, con datos de confianza
el.insertAdjacentHTML('beforeend', html)	Añade HTML al final (o al principio, o antes, o después) sin borrar lo que había	Para ir acumulando filas, tarjetas, mensajes
el.innerHTML = ''	Vacía el elemento	Antes de volver a pintar

```
const salida = document.getElementById('salida');
const nombre = 'Ada';
const edad = 36;

salida.innerHTML = `<h2>Ficha</h2><p>${nombre} tiene ${edad} años.</p>`;
salida.insertAdjacentHTML('beforeend', '<p><em>Generado por JavaScript</em></p>');
```

Las plantillas literales de la UD2 son la herramienta natural para generar HTML: admiten saltos de línea y expresiones ${…}. Para generar una lista a partir de varios valores, acumula el HTML en una cadena con un bucle y asígnalo una sola vez al final (modificar innerHTML dentro del bucle es más lento, porque el navegador repinta cada vez):

```
const lenguajes = 'HTML,CSS,JavaScript,TypeScript';
let html = '<ul>';
for (const lenguaje of lenguajes.split(',')) {
  html += `<li>${lenguaje}</li>`;
}
html += '</ul>';
salida.innerHTML = html;
```

5.3. El peligro de innerHTML: inyección de código

Si el texto que insertas con innerHTML viene del usuario (de un prompt, de un formulario, de la URL, de una API), el usuario puede meter etiquetas. Prueba con el programa anterior y escribe como nombre <img src=x onerror="alert('hackeado')">: el navegador creará esa imagen, fallará al cargarla y ejecutará el código del atributo. Es un ataque de XSS (*cross-site scripting*), uno de los más comunes de la web, y permite robar sesiones o datos de otros usuarios.

La regla es sencilla: todo lo que venga del usuario se inserta con textContent (o se escapa antes). Si necesitas combinar marcado tuyo con texto del usuario, genera el marcado con innerHTML y rellena el texto después con textContent:

```
const nombre = prompt('¿Cómo te llamas?') ?? 'anónimo';
salida.innerHTML = '<h2>Hola, <span id="quien"></span></h2>'; // marcado propio, seguro
document.getElementById('quien').textContent = nombre;       // texto del usuario, seguro
```

O escapa los cinco caracteres peligrosos con una función de utilidad que usarás durante todo el curso:

```
/**
 * Escapa los caracteres con significado en HTML para poder insertar
 * texto del usuario dentro de una plantilla de innerHTML sin riesgo de XSS.
 * @param {string} texto - Texto sin procesar.
 * @returns {string} Texto seguro para insertar como HTML.
 */
function escaparHtml(texto) {
  return String(texto)
    .replaceAll('&', '&amp;')
    .replaceAll('<', '&lt;')
    .replaceAll('>', '&gt;')
    .replaceAll('"', '&quot;')
    .replaceAll("'", '&#39;');
}

salida.innerHTML = `<h2>Hola, ${escaparHtml(nombre)}</h2>`;
```

Fíjate en que & se escapa primero: si lo hicieras al final, volverías a escapar los & de <.

5.4. document.write: por qué no

En código antiguo (y en el capítulo 1 del manual) verás document.write(' Hola '). Funciona solo mientras la página se está cargando: si se llama después (desde un temporizador, desde un script con defer, desde un clic), borra el documento entero y lo sustituye por lo que escribas. Además, bloquea el analizador del HTML. No lo uses; se menciona aquí para que lo reconozcas cuando lo veas.

5.5. El título y otros detalles del documento

Algunas propiedades de document que no necesitan la UD6:

```
document.title = 'Carrito (3)';        // cambia el texto de la pestaña
document.title;                        // y se puede leer
document.lang;                         // 'es' (del atributo lang del <html>)
document.documentElement.lang = 'en';  // cambiarlo (documentElement es el <html>)
document.body.dataset.tema = 'oscuro'; // escribe data-tema="oscuro" en el <body>
document.URL;                          // la dirección de la página (como location.href)
document.referrer;                     // de dónde venía el usuario ('' si escribió la URL)
document.visibilityState;              // 'visible' o 'hidden' (¿está la pestaña a la vista?)
```

dataset lee y escribe los atributos data-*: dataset.tema es data-tema. Lo usarás para aplicar el tema claro u oscuro: tu CSS define los colores según [data-tema="oscuro"] y JavaScript solo cambia el atributo. Es la forma correcta de que el código cambie el aspecto: tocar un atributo o una clase, y dejar los colores en la hoja de estilos.

A5. La página habla (CE c, d, h)

Objetivo: generar texto y HTML desde código de forma segura, y empezar a aplicar los datos del proyecto.

Crea ut03/a5-pagina con el index.html de partida y una hoja estilos.css mínima (al menos una regla para [data-tema="oscuro"] que cambie los colores de fondo y texto del body).

1. Tres formas de escribir. Inserta el texto Hola en el <main> con textContent y luego con innerHTML, y después añade un párrafo con insertAdjacentHTML('beforeend', …) sin borrar lo anterior. Haz una captura de las tres y explica en NOTAS.md la diferencia.
2. Ficha del proyecto. Con los datos que ya tenías en variables en la A9 de la UD2 (o con cuatro o cinco propiedades de un elemento de tu colección), genera con una plantilla literal una «tarjeta» en HTML (<article> con un <h2> y una lista de definición <dl>) y píntala en el <main>. Formatea con Intl cualquier fecha o importe que contenga.
3. Lista generada. Escribe en una cadena separada por comas los nombres de cinco elementos de tu colección y genera una <ul> con un <li> por elemento, acumulando el HTML en un bucle y asignándolo una sola vez.
4. El ataque. Pide con prompt el nombre del usuario y salúdalo con innerHTML dentro de un <h2>. Escribe como nombre <img src=x onerror="alert('XSS')"> y comprueba qué pasa. Corrígelo de las dos formas del apartado 5.3 (en dos ficheros distintos) y comprueba que el ataque ya no funciona. Incluye escaparHtml documentada con JSDoc.
5. Tema y título. Añade un botón <button id="tema">Cambiar tema</button> al HTML. Para que haga algo necesitas una línea de la UD7, que de momento copias tal cual: document.getElementById('tema').addEventListener('click', () => { … }); («cuando hagan clic, ejecuta esto»). Dentro, alterna document.body.dataset.tema entre 'claro' y 'oscuro' y actualiza document.title con el tema activo.

Entrega: la carpeta a5-pagina con un fichero por punto, estilos.css y NOTAS.md.

6. Hablar con el navegador: el BOM

El **BOM** (*Browser Object Model*) es el conjunto de objetos con los que el navegador te deja consultar y controlar la ventana, la dirección, el historial, el dispositivo y la pantalla. Cuelgan todos de window.

Objeto	Para qué sirve	Lo más usado
window	La ventana o pestaña; tamaño, desplazamiento, abrir otras ventanas	innerWidth, innerHeight, scrollTo, open, matchMedia
location	La dirección (URL) de la página; leerla y navegar	href, pathname, search, hash, assign, reload
history	El historial de la pestaña	back, forward, length, pushState
navigator	El navegador y el dispositivo	language, onLine, userAgent, clipboard, geolocation, share
screen	La pantalla física	width, height, orientation

6.1. location: leer y cambiar la dirección

Para la URL http://127.0.0.1:5500/ut03/a6-bom/index.html?pagina=2&orden=titulo#resultados:

Propiedad	Valor
location.href	la URL completa
location.protocol	'http:'
location.host	'127.0.0.1:5500' (hostname sin el puerto, port solo el puerto)
location.origin	'http://127.0.0.1:5500' — el **origen**: protocolo + host + puerto
location.pathname	'/ut03/a6-bom/index.html'
location.search	'?pagina=2&orden=titulo' — la cadena de consulta
location.hash	'#resultados' — el fragmento

El origen es el concepto de seguridad más importante del navegador: dos páginas solo comparten almacenamiento, cookies y pueden comunicarse libremente si tienen el mismo origen. Volverá en el apartado 8 y en la UD8 (CORS).

Para leer los parámetros de la URL no analices location.search a mano: usa URLSearchParams.

```
const params = new URLSearchParams(location.search);
params.get('pagina');        // '2'  (siempre texto, o null si no existe)
params.has('orden');         // true
Number(params.get('pagina') ?? 1); // 2, con valor por defecto si falta

// Y para construir una URL con parámetros, al revés:
const nuevos = new URLSearchParams({ pagina: 3, orden: 'fecha' });
nuevos.toString();           // 'pagina=3&orden=fecha'
```

Para navegar desde código:

```
location.href = 'detalle.html?id=7';  // navega (queda en el historial)
location.assign('detalle.html?id=7'); // lo mismo
location.replace('login.html');       // navega SIN dejar la página actual en el historial
location.reload();                    // recarga
location.hash = 'resultados';         // salta al elemento con id="resultados" sin recargar
```

6.2. history: moverse por el historial

```
history.length;      // cuántas entradas hay en esta pestaña
history.back();      // como el botón Atrás
history.forward();   // como Adelante
history.go(-2);      // dos páginas atrás
```

Por privacidad, no puedes leer las direcciones del historial, solo moverte por él. history.pushState(estado, '', 'nueva-url') añade una entrada sin recargar la página: es la base de las *single page applications* y lo que hace por ti el enrutador de Angular en la UD9.

6.3. navigator: qué navegador, qué dispositivo, qué capacidades

```
navigator.language;     // 'es-ES': idioma preferido (apartado 3.4)
navigator.languages;    // ['es-ES', 'es', 'en']: todos, por orden de preferencia
navigator.onLine;       // true/false: ¿hay conexión?
navigator.userAgent;    // texto largo con el navegador y el sistema... poco fiable
navigator.hardwareConcurrency; // núcleos de CPU
```

```
navigator.maxTouchPoints;     // > 0 en pantallas táctiles
navigator.cookieEnabled;      // ¿acepta cookies?
```

Y varias API que piden permiso al usuario o que requieren un origen seguro (https:// o localhost/127.0.0.1), que tienen en común que son asíncronas (devuelven una promesa, que estudiarás en la UD8). Puedes usarlas ya con la forma .then(resultado => …):

```
// Portapapeles (requiere un gesto del usuario, como un clic)
navigator.clipboard.writeText('Texto copiado').then(() => console.log('Copiado'));

// Geolocalización (pide permiso)
navigator.geolocation.getCurrentPosition(
  (pos) => console.log(pos.coords.latitude, pos.coords.longitude),
  (err) => console.warn('Sin posición:', err.message),
);

// Compartir con el sistema (móviles y algunos escritorios)
if (navigator.share) {
  navigator.share({ title: document.title, url: location.href });
}
```

No uses userAgent para decidir qué hacer («si es Chrome, entonces…»): es fácil de falsear y los navegadores lo han ido vaciando de información. Comprueba si existe la capacidad que necesitas (if (navigator.share)), como en el apartado 1.5. Esta técnica se llama detección de características.

6.4. screen, window y la apariencia de la ventana

```
screen.width, screen.height;          // pantalla física, en píxeles CSS
screen.orientation.type;              // 'landscape-primary', 'portrait-primary'…
innerWidth, innerHeight;              // zona visible de la página (viewport)
outerWidth, outerHeight;              // la ventana del navegador entera
scrollY;                              // cuántos píxeles se ha desplazado la página
scrollTo({ top: 0, behavior: 'smooth' });   // volver arriba con animación
```

Sobre la apariencia de la ventana, el navegador te deja hacer menos de lo que el criterio de evaluación sugiere, y es deliberado: los sitios web no pueden mover, redimensionar ni cerrar la ventana del usuario (solo ventanas que ellos mismos abrieron, apartado 8). Lo que sí puedes controlar:

- El título de la pestaña (document.title) y el favicon (cambiando el href del <link rel="icon">): es la forma habitual de avisar de novedades («(3) Mensajes»).
- La pantalla completa: document.documentElement.requestFullscreen() y document.exitFullscreen(), siempre a petición del usuario (desde un clic).
- El color de la barra del navegador en móviles, con <meta name="theme-color">.
- El tema de tu propia página, respetando la preferencia del sistema operativo con matchMedia:

```
const prefiereOscuro = matchMedia('(prefers-color-scheme: dark)').matches; // true/false
document.body.dataset.tema = prefiereOscuro ? 'oscuro' : 'claro';
```

matchMedia evalúa una consulta de medios de CSS desde JavaScript y devuelve un objeto cuyo matches dice si se cumple ahora. En el proyecto, la lógica será: si el usuario eligió un tema, se usa el guardado; si no, el del sistema.

A6. Radiografía del navegador (CE b, c, h)

Objetivo: analizar los objetos del BOM y usarlos para leer la URL y adaptar la página al navegador del usuario.

Crea ut03/a6-bom con el index.html de partida y una hoja de estilos con tema claro y oscuro.

1. Panel de información. Genera en el <main> una tabla HTML (<table>) con estas filas, leídas del BOM: idioma, idiomas, conexión, núcleos de CPU, pantalla táctil, tamaño de la pantalla, tamaño de la zona visible, orientación, origen de la página, ruta y si prefiere el tema oscuro. Usa textContent o escaparHtml para los valores.
2. Parámetros de la URL. Lee con URLSearchParams los parámetros nombre y tema. Si viene nombre, salúdalo en un <h2> (de forma segura: pruébalo con una etiqueta). Si viene tema (claro u oscuro), aplícalo con dataset.tema; si no viene, usa el del sistema con matchMedia. Prueba index.html?nombre=Ada&tema=oscuro.
3. Navegación. Añade tres botones: «Añadir parámetro» (navega con location.href a la misma página con pagina incrementado en uno, construyendo la cadena con URLSearchParams), «Atrás» (history.back()) y «Recargar». Muestra history.length y comprueba cómo crece. Añade un cuarto que use location.replace y explica en NOTAS.md qué cambia en el botón Atrás del navegador.
4. Pantalla completa y título. Un botón que ponga la página en pantalla completa y otro que la saque; mientras está en pantalla completa, el document.title debe empezar por [PC]. Comprueba qué ocurre si llamas a requestFullscreen() sin un clic (desde el script directamente) y anótalo.
5. Detección de características. Comprueba con if si existen navigator.share, navigator.clipboard y navigator.geolocation, y muestra en el panel «disponible» o «no disponible» para cada una. Añade un botón «Copiar URL» que copie location.href al portapapeles (solo si la API existe) y confirme en la página que se ha copiado.

Entrega: la carpeta a6-bom y NOTAS.md con las respuestas de los puntos 3 y 4.

7. Hablar con el usuario: diálogos

7.1. Los tres diálogos del navegador

Ya conoces prompt. Tiene dos hermanos:

```
alert('Guardado correctamente');              // informa; no devuelve nada
const seguro = confirm('¿Eliminar el elemento?');  // true (Aceptar) o false (Cancelar)
const nombre = prompt('Nombre del elemento', 'Sin título'); // texto, '' o null (Cancelar)
```

Los tres bloquean la página: nada se ejecuta (ni temporizadores, ni animaciones) hasta que el usuario responde. Su aspecto lo decide el navegador y no se puede cambiar; algunos navegadores permiten al usuario silenciarlos («no mostrar más diálogos»), y entonces confirm devuelve false y prompt devuelve null sin preguntar. Son útiles para aprender, depurar y para prototipos; en una aplicación real se sustituyen por diálogos propios.

Lo que sí debes hacer siempre con ellos es tratar la cancelación: prompt devuelve null si se cancela y '' si se acepta vacío, y son casos distintos. En la UD2 aprendiste a validarlo; aquí lo seguimos aplicando.

7.2. Diálogos propios con <dialog>

HTML tiene un elemento <dialog> que el navegador sabe abrir como ventana modal (bloquea el resto de la página, se cierra con Esc, atrapa el foco, es accesible) y cuyo aspecto controlas tú con CSS:

```
<dialog id="confirmar">
  <h2>¿Eliminar el elemento?</h2>
  <p>Esta acción no se puede deshacer.</p>
  <form method="dialog">
    <button value="cancelar">Cancelar</button>
```

```
  <button value="eliminar">Eliminar</button>
 </form>
</dialog>
<button id="borrar">Eliminar</button>
const dialogo = document.getElementById('confirmar');

document.getElementById('borrar').addEventListener('click', () => {
 dialogo.showModal();              // lo abre como modal
});

dialogo.addEventListener('close', () => {
 // returnValue es el value del botón que cerró el diálogo
 if (dialogo.returnValue === 'eliminar') {
  console.log('Eliminado');
 } else {
  console.log('Cancelado');
 }
});
```

Las piezas clave:

- dialogo.showModal() abre el diálogo de forma modal; dialogo.show() lo abre sin bloquear el resto; dialogo.close(valor) lo cierra desde código.
- Un <form method="dialog"> dentro del diálogo lo cierra al enviarse, **sin** recargar la página, y guarda en dialogo.returnValue el value del botón pulsado. No necesitas JavaScript para cerrar.
- El evento close avisa de que se ha cerrado (por un botón, por Esc o por close()), y ahí lees returnValue.
- Con CSS, dialog::backdrop da estilo al fondo oscurecido.

Fíjate en que, a diferencia de confirm, el código no espera: showModal() abre el diálogo y sigue; la respuesta llega después, en el evento close. Es la misma idea de los temporizadores (apartado 4): entregas una función y el navegador la llama cuando toca. Toda la interacción con el usuario funciona así, y la UD7 está dedicada a ello.

Para pedir un dato (el equivalente a prompt), pon un <input> dentro del formulario del diálogo y léelo en el evento close:

```
<dialog id="pedirNombre">
 <form method="dialog">
  <label>Nombre <input id="nombre" required></label>
  <button value="ok">Aceptar</button>
 </form>
</dialog>
const d = document.getElementById('pedirNombre');
d.addEventListener('close', () => {
 if (d.returnValue === 'ok') {
  console.log(document.getElementById('nombre').value);
 }
});
d.showModal();
```

7.3. Otras formas de avisar

- Mensajes en la propia página (una zona <p id="mensaje" role="status"> que rellenas con textContent): es la forma más sencilla y la más accesible para confirmaciones breves («Guardado»). Combinada con un setTimeout que lo borra a los tres segundos, es el clásico *toast*.
- Notificaciones del sistema (Notification.requestPermission() y new Notification('Título', { body })): aparecen fuera de la página, requieren permiso y un origen seguro, y se usan poco en aplicaciones de escritorio dentro del navegador. Las dejamos como ampliación.

A7. Diálogos propios (CE e, h)

Objetivo: interactuar con el usuario con los diálogos del navegador y sustituirlos por diálogos propios con <dialog>.

Crea ut03/a7-dialogos con el index.html de partida.

1. Los tres de siempre. Un programa que, al cargar, pide un nombre con prompt (con valor por defecto), confirma con confirm si quiere continuar y termina con alert. Trata los tres casos de prompt (texto, vacío, cancelación) mostrando mensajes distintos en la página. Comprueba qué devuelven confirm y prompt si marcas en el navegador «no permitir más diálogos» (Chrome lo ofrece tras varios seguidos).
2. Confirmar con <dialog>. Un botón «Vaciar lista» que abre un diálogo modal de confirmación como el del apartado 7.2. Si se confirma, vacía el <main> y muestra un mensaje de estado; si se cancela (botón o Esc), no hace nada. Da estilo al diálogo y a ::backdrop en estilos.css.
3. Pedir un dato con <dialog>. Un botón «Añadir» que abre un diálogo con un <input> para el nombre de un elemento de tu colección y, al aceptar, lo añade como <li> a una lista en la página (de forma segura). Al cancelar, no añade nada. Comprueba que required impide aceptar con el campo vacío.
4. Toast. Una función avisar(texto) que escribe el texto en un <p id="mensaje" role="status"> y lo borra a los tres segundos; si se llama dos veces seguidas, debe cancelar el borrado anterior (guarda el identificador del setTimeout en una variable externa). Úsala en los puntos 2 y 3.
5. Ampliación · Notificación. Un botón que pide permiso para notificaciones y, si se concede, muestra una notificación del sistema con el título de la página.

Entrega: la carpeta a7-dialogos con estilos.css y las respuestas del punto 1 en NOTAS.md.

8. Varias ventanas: abrir, cerrar y comunicar

8.1. Abrir ventanas y pestañas

```
// Pestaña nueva (lo habitual)
const pestana = open('https://developer.mozilla.org', '_blank');

// Ventana emergente con tamaño y posición
const popup = open('vista-previa.html', 'previa', 'width=480,height=640,left=100,top=100');

// Volver a usar el mismo nombre reutiliza la ventana en lugar de abrir otra
open('vista-previa.html?id=2', 'previa');
```

window.open(url, nombre, características) devuelve una referencia a la nueva ventana (su propio objeto window), o null si el navegador la bloqueó. Y la bloqueará casi siempre que la abras sin un gesto del usuario: los bloqueadores de ventanas emergentes permiten open dentro de un clic y lo impiden desde un temporizador o al cargar la página. Comprueba siempre el resultado:

```
const nueva = open('ayuda.html', '_blank');
if (nueva === null) {
  avisar('El navegador ha bloqueado la ventana. Permite las ventanas emergentes para este sitio.');
}
```

Con la referencia puedes controlar esa ventana (es tuya): nueva.close(), nueva.focus(), nueva.closed (si el usuario ya la cerró), e incluso nueva.document si es del mismo origen. Y en sentido inverso, la ventana abierta encuentra a quien la abrió en window.opener. Por seguridad, cuando abras un sitio ajeno corta esa relación con 'noopener' en las características (o rel="noopener" en los enlaces), para que la otra página no pueda manipular la tuya.

Si cada actividad en un enlace que abre otra pestaña te suena a <a href target="_blank">, es que ya sabes lo esencial: open es la versión programática.

8.2. Comunicar ventanas: postMessage

Dos ventanas (una página y su emergente, una página y un <iframe>) se envían mensajes con postMessage, aunque sean de orígenes distintos. El emisor indica a qué origen va dirigido el mensaje y el receptor comprueba de qué origen viene. Es la única forma segura de comunicar páginas de distintos sitios.

```
// En la ventana principal: abrir y enviar
const previa = open('previa.html', 'previa', 'width=480,height=640');
setTimeout(() => {
  previa.postMessage({ tipo: 'ficha', titulo: 'Dune', precio: 12.5 }, location.origin);
}, 1000); // damos tiempo a que cargue (en la UD7 lo haremos bien, con el evento load)

// En previa.html: recibir
addEventListener('message', (evento) => {
  if (evento.origin !== location.origin) {
    return; // ignoramos mensajes de otros sitios
  }
  console.log('Recibido de', evento.origin, evento.data);
  document.getElementById('salida').textContent = `${evento.data.titulo}: ${evento.data.precio} €`;
  // Responder a quien nos abrió:
  evento.source.postMessage({ tipo: 'ok' }, evento.origin);
});
```

El segundo argumento de postMessage es el origen de destino: location.origin si es tu propio sitio, el origen concreto del otro sitio si es ajeno. Nunca '*' con datos sensibles. En el receptor, evento.origin identifica al emisor y evento.data es el mensaje (puede ser cualquier valor serializable: texto, número, objeto…).

8.3. Comunicar pestañas del mismo origen: BroadcastChannel y el evento storage

Si el usuario tiene tu aplicación abierta en dos pestañas y cambia el tema en una, lo esperable es que la otra también cambie. Para eso no hay referencia entre ventanas (la segunda pestaña no la abrió la primera), pero hay dos mecanismos que funcionan entre todas las pestañas del mismo origen:

```
// BroadcastChannel: un «canal de radio» con nombre
const canal = new BroadcastChannel('preferencias');

canal.postMessage({ tema: 'oscuro' });          // emite a las demás pestañas (no a sí misma)

canal.addEventListener('message', (evento) => {   // recibe en las demás
```

```
  document.body.dataset.tema = evento.data.tema;
});
```

El segundo mecanismo lo verás en el apartado 9: cuando una pestaña escribe en localStorage, las demás pestañas del mismo origen reciben un evento storage con la clave y el valor nuevo. Es un efecto secundario del almacenamiento que resulta muy práctico: guardas la preferencia y, de paso, avisas al resto.

Mecanismo	Entre quiénes	¿Distintos orígenes?	Cuándo usarlo
Referencia (open → opener)	Una ventana y la que abrió	Solo mismo origen para acceder al contenido	Ventanas auxiliares (vista previa, selector)
postMessage	Dos ventanas con referencia o un <iframe>	Sí, con comprobación de origen	Integrar contenido de otro sitio
BroadcastChannel	Todas las pestañas del mismo origen	No	Sincronizar estado entre pestañas
Evento storage	Todas las pestañas del mismo origen	No	Sincronizar cuando el estado ya se guarda en localStorage

A8. Ventanas que se hablan (CE b, f, h)

Objetivo: abrir y controlar ventanas, y comunicar ventanas y pestañas con postMessage y BroadcastChannel.

Crea ut03/a8-ventanas con dos páginas: index.html (principal) y previa.html (secundaria), cada una con su script.

1. Abrir y controlar. En la principal, tres botones: «Abrir vista previa» (abre previa.html como ventana emergente de 480 × 640 con el nombre previa), «Enfocar» (focus() sobre la referencia) y «Cerrar» (close()). Muestra en la página si la ventana está abierta o cerrada (closed), actualizándolo cada segundo con setInterval. Intenta abrirla desde un setTimeout al cargar la página y anota en NOTAS.md qué hace el navegador.
2. Enviar una ficha. Un botón «Enviar» que manda a la vista previa, con postMessage, un objeto con el título, el precio y la fecha de un elemento de tu colección. La vista previa lo muestra formateado con Intl (de forma segura) y responde con un mensaje de confirmación que la principal escribe en su zona de estado. Comprueba el origen en ambos lados.
3. Origen equivocado. Cambia en el emisor el origen de destino por 'http://localhost:5500' (si trabajas en 127.0.0.1) y observa en la consola qué ocurre. Restáuralo y explica en NOTAS.md por qué el navegador se ha negado.
4. Tema sincronizado. Abre index.html en dos pestañas. Un botón «Cambiar tema» que alterna dataset.tema y lo emite por un BroadcastChannel llamado preferencias; la otra pestaña debe cambiar sola. Comprueba que la pestaña emisora no recibe su propio mensaje.
5. Ampliación · noopener. Abre https://developer.mozilla.org con y sin 'noopener' y comprueba en la consola de la nueva pestaña el valor de window.opener en cada caso.

Entrega: la carpeta a8-ventanas con las dos páginas y NOTAS.md.

9. Recordar cosas: almacenamiento en el navegador

Hasta ahora, cada recarga borraba todo: las variables de tu programa viven en memoria y mueren con la página. El navegador ofrece varios mecanismos para guardar información en el equipo del usuario y recuperarla en la siguiente visita. Todos están ligados al origen de la página (apartado 6.1): lo que guarda http://127.0.0.1:5500 no lo ve http://localhost:5500 ni ningún otro sitio.

Mecanismo	Qué guarda	Capacidad	Dura	Se envía al servidor	Ámbito
Cookies	Texto corto (pares clave=valor)	~4 KB por cookie, unas 50 por sitio	Lo que indique expires/max-age; sin ellos, hasta cerrar el navegador	Sí, en cada petición	Dominio y ruta
localStorage	Pares clave → texto	~5–10 MB por origen	Hasta que se borre (por código o por el usuario)	No	Origen
sessionStorage	Pares clave → texto	~5 MB	Hasta cerrar la pestaña	No	Origen y pestaña
IndexedDB	Objetos estructurados, ficheros	Cientos de MB o más	Hasta que se borre	No	Origen
Cache Storage	Respuestas HTTP (para funcionar sin conexión)	Grande	Hasta que se borre	No	Origen

En esta unidad trabajarás a fondo con las tres primeras filas y conocerás IndexedDB; Cache Storage pertenece a las aplicaciones web progresivas y queda fuera del módulo.

9.1. Cookies

Las cookies son el mecanismo más antiguo (1994) y el único que viaja al servidor en cada petición: por eso sirven para mantener sesiones iniciadas, y por eso son el centro de la normativa de privacidad (apartado 10). Desde JavaScript se manejan con document.cookie, que tiene una interfaz incómoda: al leer devuelve todas las cookies en una sola cadena; al escribir añade o modifica una cookie con sus atributos.

```
// Escribir: nombre=valor; atributos separados por punto y coma
document.cookie = 'tema=oscuro; max-age=31536000; path=/; SameSite=Lax';
document.cookie = 'idioma=es-ES; max-age=31536000; path=/; SameSite=Lax';

// Leer: una única cadena con TODAS las cookies accesibles
console.log(document.cookie); // 'tema=oscuro; idioma=es-ES'

// Borrar: la misma cookie con caducidad cero
document.cookie = 'tema=; max-age=0; path=/';
```

Atributo	Significado
max-age=segundos	Cuánto dura (alternativa moderna a expires=fecha). Sin él, la cookie es de sesión: desaparece al cerrar el navegador
path=/	En qué rutas del sitio se envía. Pon siempre / para que sea visible en todo el sitio
domain=	A qué dominio pertenece (por defecto, el actual sin subdominios)
Secure	Solo se envía por HTTPS
SameSite=Lax / Strict / None	Si se envía en peticiones que vienen de otros sitios. Lax es el valor por defecto razonable; None requiere Secure
HttpOnly	Inaccesible desde JavaScript (solo la pone el servidor). Es la protección de las cookies de sesión frente al XSS del apartado 5.3

Como leer una cookie concreta obliga a trocear la cadena, se escribe una pequeña función de utilidad (la tienes en las soluciones y la usarás en la A9):

```
/**
 * Devuelve el valor de una cookie o null si no existe.
 * @param {string} nombre
 * @returns {string | null}
 */
function leerCookie(nombre) {
 for (const par of document.cookie.split('; ')) {
  const [clave, valor] = par.split('=');  // desestructuración: lo verás en la UD4
  if (clave === nombre) {
   return decodeURIComponent(valor);
  }
 }
 return null;
}
```

Los valores con espacios, punto y coma o caracteres no ASCII deben codificarse con encodeURIComponent al escribir y decodificarse al leer. Hay una API moderna más cómoda (cookieStore, asíncrona), pero todavía no está en todos los navegadores.

¿Cuándo usar cookies desde el cliente? Casi nunca: si el dato no necesita viajar al servidor, localStorage es más sencillo, más grande y no engorda cada petición. Las cookies de sesión las pone el servidor (lo verás en el módulo de entorno servidor).

9.2. localStorage y sessionStorage

Los dos tienen la misma API, sencillísima, y solo se diferencian en cuánto duran y en si se comparten entre pestañas:

```
localStorage.setItem('tema', 'oscuro');      // guardar (clave y valor, ambos texto)
localStorage.getItem('tema');              // 'oscuro'  (o null si no existe)
localStorage.removeItem('tema');           // borrar una clave
localStorage.clear();                    // borrar todo lo de este origen
localStorage.length;                     // cuántas claves hay
localStorage.key(0);                     // nombre de la primera clave
```

- localStorage persiste indefinidamente y lo comparten todas las pestañas y ventanas del mismo origen. Es el sitio para las preferencias y, en la UD5, para los datos del proyecto.
- sessionStorage vive solo en esa pestaña: al cerrarla se borra, y otra pestaña del mismo sitio tiene el suyo propio. Sirve para el estado de un proceso en curso (un formulario a medias, el paso de un asistente, un filtro aplicado).

Dos detalles importantes:

Solo guarda texto. Si guardas un número, recuperas una cadena (localStorage.setItem('n', 42); localStorage.getItem('n') // '42'). Si guardas un objeto, recuperas '[object Object]'. Para guardar cualquier cosa que no sea texto, conviértela a JSON al guardar y vuelve a convertirla al leer:

```
const preferencias = { tema: 'oscuro', idioma: 'es-ES', moneda: 'EUR', porPagina: 10 };

// Guardar: objeto → texto JSON
localStorage.setItem('preferencias', JSON.stringify(preferencias));
// En Application → Local Storage verás: {"tema":"oscuro","idioma":"es-
ES","moneda":"EUR","porPagina":10}
```

```
// Leer: texto JSON → objeto (o null si no había nada)
const guardadas = JSON.parse(localStorage.getItem('preferencias'));
console.log(guardadas?.tema); // 'oscuro' (?. evita el error si guardadas es null)
```

JSON (*JavaScript Object Notation*) es el formato de texto con el que se intercambian datos en la web: lo usarás con las API en la UD8 y es lo que hay dentro de package.json o angular.json. JSON.stringify convierte un valor a texto JSON y JSON.parse hace lo contrario. Dos cosas que JSON no admite: comentarios y comillas simples. Y una que conviene saber: las fechas se guardan como texto ISO y hay que reconstruirlas con new Date(texto) al leer.

Puede fallar. Si el usuario navega en modo privado con el almacenamiento desactivado o si se supera la cuota, setItem lanza una excepción. Y si alguien ha editado a mano el valor guardado, JSON.parse puede fallar. Protege ambas operaciones con try…catch (lo viste en la UD2 como forma de capturar errores):

```
/**
 * Guarda un valor cualquiera en localStorage como JSON.
 * @param {string} clave
 * @param {unknown} valor
 * @returns {boolean} true si se ha guardado.
 */
function guardar(clave, valor) {
  try {
    localStorage.setItem(clave, JSON.stringify(valor));
    return true;
  } catch (error) {
    console.error('No se pudo guardar', clave, error);
    return false;
  }
}

/**
 * Lee un valor guardado como JSON; si no existe o está corrupto, devuelve porDefecto.
 * @param {string} clave
 * @param {unknown} porDefecto
 * @returns {unknown}
 */
function cargar(clave, porDefecto) {
  try {
    const texto = localStorage.getItem(clave);
    return texto === null ? porDefecto : JSON.parse(texto);
  } catch (error) {
    console.warn('Valor corrupto en', clave, error);
    return porDefecto;
  }
}
```

Estas dos funciones son el embrión del repositorio que construirás en la UD5. Guárdalas en un fichero aparte (almacen.js) y reutilízalas.

9.3. El evento storage

Cuando una pestaña modifica localStorage, las demás pestañas del mismo origen reciben un evento storage (la que escribió, no). Trae la clave, el valor antiguo y el nuevo:

```
addEventListener('storage', (evento) => {
 if (evento.key === 'preferencias') {
  const nuevas = JSON.parse(evento.newValue);
  document.body.dataset.tema = nuevas.tema;
 }
});
```

Es la alternativa a BroadcastChannel del apartado 8.3 cuando lo que sincronizas ya se guarda: sin código extra, todas las pestañas se enteran.

9.4. IndexedDB: cuando localStorage se queda pequeño

localStorage es síncrono (bloquea la página mientras lee o escribe), solo guarda texto y no permite buscar: para encontrar un elemento tienes que cargarlo todo. Para colecciones grandes (miles de registros, ficheros, imágenes) el navegador ofrece IndexedDB: una base de datos de objetos, con almacenes (equivalentes a tablas), índices para buscar, transacciones y una API asíncrona que no bloquea la página.

Su API nativa es verbosa (todo se hace con peticiones y eventos de éxito y error). Esto es lo mínimo para ver cómo funciona:

```
const peticion = indexedDB.open('dwec', 1);

// Solo la primera vez (o al subir la versión): crear la estructura
peticion.addEventListener('upgradeneeded', () => {
 const bd = peticion.result;
 bd.createObjectStore('elementos', { keyPath: 'id' });
});

peticion.addEventListener('success', () => {
 const bd = peticion.result;
 const tx = bd.transaction('elementos', 'readwrite');
 const almacen = tx.objectStore('elementos');
 almacen.put({ id: 1, titulo: 'Dune', precio: 12.5 });  // guardar un objeto entero
 const lectura = almacen.get(1);
 lectura.addEventListener('success', () => console.log(lectura.result));
});
```

Fíjate en que no hay JSON.stringify: IndexedDB guarda objetos tal cual (incluidas fechas). En la práctica se usa a través de una librería que la envuelve con promesas (idb o idb-keyval), y en el proyecto podrás elegirla en lugar de localStorage en la UD5 si tu colección lo pide (R6 admite ambas). En esta unidad basta con que sepas qué es, para qué sirve y cómo se ve en las DevTools.

9.5. Verlo todo: la pestaña Application

En las DevTools, la pestaña Application (en Firefox, Almacenamiento) muestra, para el origen actual, las cookies, localStorage, sessionStorage, IndexedDB y Cache Storage. Puedes ver cada clave y su valor, editarlos a mano (doble clic), borrarlos uno a uno o con Clear site data, y ver cómo cambian en tiempo real mientras tu programa escribe. Es la herramienta de depuración de esta parte de la unidad: antes de sospechar de tu código, mira qué hay guardado de verdad.

> *Consejo. Añade una versión a lo que guardas ({ version: 1, tema: … } o una clave preferencias.v1). Cuando en una unidad posterior cambies la estructura, podrás detectar los datos antiguos y migrarlos o descartarlos en lugar de que la aplicación falle con lo que guardó una versión anterior.*

A9. Guardar y recuperar (CE g, h)

Objetivo: usar cookies, localStorage y sessionStorage, elegir el adecuado en cada caso y depurar con la pestaña Application.

Crea ut03/a9-almacen con el index.html de partida y almacen.js con las funciones guardar y cargar del apartado 9.2 (enlázalo antes que app.js).

1. Contador de visitas, tres veces. Guarda y muestra «Has visitado esta página N veces» usando una cookie (con max-age de un año), localStorage y sessionStorage, los tres a la vez con claves distintas. Recarga varias veces, abre la página en otra pestaña y después cierra y abre el navegador. Anota en NOTAS.md qué contador cambia en cada caso y por qué.
2. Cookies con atributos. Crea una cookie de sesión (sin max-age) y otra con max-age=60. Obsérvalas en Application → Cookies (columnas Expires y Size), espera un minuto y recarga. Intenta leer desde JavaScript una cookie marcada como HttpOnly (puedes crearla a mano en Application) y anota qué ves en document.cookie.
3. Guardar un objeto. Guarda en localStorage un objeto preferencias con cuatro propiedades (una de ellas numérica y otra booleana) con guardar, recupéralo con cargar y muestra en la página cada propiedad con su typeof. Después guárdalo sin JSON.stringify y mira qué aparece en Application; recupéralo y explica en NOTAS.md qué has obtenido.
4. Datos corruptos. Edita a mano en Application el valor de preferencias para que deje de ser JSON válido (borra una llave) y recarga. Comprueba que cargar devuelve el valor por defecto y que la página no se rompe. Quita el try…catch y comprueba qué pasa; vuelve a ponerlo.
5. Dos pestañas. Añade un addEventListener('storage', …) que escriba en la página «La otra pestaña ha cambiado la clave X» cuando cambie algo. Abre dos pestañas y modifica las preferencias en una.
6. Ampliación · IndexedDB. Ejecuta el ejemplo del apartado 9.4, localiza la base de datos en Application → IndexedDB y añade un segundo objeto a mano con put desde la consola.

Entrega: la carpeta a9-almacen con almacen.js, un fichero por punto y NOTAS.md.

10. Privacidad: lo que la ley y el sentido común exigen

10.1. Qué es un dato personal y qué tiene que ver contigo

El Reglamento General de Protección de Datos (RGPD, UE 2016/679) y la Ley Orgánica 3/2018 (LOPDGDD) regulan el tratamiento de datos personales: cualquier información sobre una persona identificada o identificable. Un nombre, un correo, una dirección IP, un identificador guardado en una cookie que permita reconocer al usuario entre visitas… todo eso son datos personales. Y guardar información en el equipo del usuario está regulado además por la Ley de Servicios de la Sociedad de la Información (LSSI, art. 22.2), que es la que exige el famoso aviso de cookies.

Como programador o programadora del lado cliente, tú eres quien escribe el código que guarda. Lo que debes saber:

- Lo que guardas sin consentimiento debe ser estrictamente necesario para el servicio que el usuario ha pedido: la cookie de sesión tras iniciar sesión, el carrito de la compra, la preferencia de idioma que él mismo ha elegido. Son las cookies y almacenamientos técnicos o funcionales, y están exentos del consentimiento.

- Todo lo demás (analítica, publicidad, seguimiento entre sitios, perfiles) requiere consentimiento previo, informado, libre y específico: el usuario debe poder rechazar con la misma facilidad que aceptar, y nada de eso puede guardarse antes de que acepte. La Agencia Española de Protección de Datos (AEPD) publica una *Guía sobre el uso de las cookies* que concreta cómo debe ser el aviso.
- Da igual que sea una cookie o localStorage: la norma habla de «dispositivos de almacenamiento y recuperación de datos en equipos terminales», es decir, cualquier mecanismo del apartado 9.
- El usuario tiene derecho a saber qué guardas, a acceder a ello, a rectificarlo y a borrarlo. En una aplicación cliente, eso se traduce en una página de preferencias que muestra lo guardado y un botón para borrarlo todo.

10.2. Buenas prácticas en el código

- Minimiza: guarda solo lo que necesitas y durante el tiempo necesario (pon max-age a las cookies; borra sessionStorage cuando el proceso termina).
- Nunca guardes contraseñas ni datos sensibles (salud, ideología, datos bancarios) en el navegador: localStorage es legible por cualquier script de la página (recuerda el XSS) y por cualquiera que se siente en ese ordenador.
- Informa: un texto breve junto a la opción («Guardaremos tus preferencias en este navegador») vale más que un aviso genérico.
- Respeta las señales del navegador: navigator.globalPrivacyControl (y el antiguo doNotTrack) indican que el usuario no quiere seguimiento.
- Versiona y valida lo que lees: nunca confíes ciegamente en lo guardado (apartado 9.5).

10.3. El aviso de consentimiento del proyecto

Las preferencias de tu proyecto (tema, idioma, moneda, elementos por página) son funcionales: las elige el usuario y existen para servirle a él. Estrictamente no necesitan consentimiento, pero el proyecto incluye el aviso porque en la UD8 añadirás llamadas a servicios externos y porque saber construirlo bien es un requisito profesional. El patrón que aplicarás en la A10:

1. Al cargar, se comprueba si existe la clave consentimiento en localStorage.
2. Si no existe, se muestra un aviso (un <dialog> no modal o una franja fija) con dos botones igual de visibles: «Aceptar» y «Rechazar», y un enlace a un texto que explica qué se guarda.
3. Si acepta, se guarda consentimiento = 'aceptado' con la fecha y, a partir de ahí, se guardan las preferencias.
4. Si rechaza, se guarda consentimiento = 'rechazado' (esa única clave es necesaria para no volver a preguntar) y las preferencias se aplican solo durante la sesión, sin persistir.
5. En la página de preferencias hay siempre un botón «Borrar mis datos» que ejecuta localStorage.clear() y vuelve al estado inicial.

11. Depurar y documentar lo que no se ve

11.1. Depurar temporizadores y almacenamiento

- Un punto de ruptura dentro de la función de un setInterval se detiene en cada tic: útil para ver los valores una vez; después quítalo o conviértelo en un punto de ruptura condicional (clic derecho sobre la línea → *Add conditional breakpoint*, por ejemplo restantes === 1).
- console.time('etiqueta') y console.timeEnd('etiqueta') miden cuánto tarda un fragmento. Úsalos para comparar innerHTML dentro y fuera de un bucle.

- En Application puedes editar y borrar claves mientras la página está abierta. Si tu programa «no guarda», comprueba primero si está guardando en otro origen (localhost frente a 127.0.0.1, o un puerto distinto de Live Server).
- En Sources → Event Listener Breakpoints puedes pedir que el depurador se detenga en cualquier temporizador (*Timer → setInterval fired*) o al recibir un mensaje de otra ventana.
- Para los errores de ventanas emergentes bloqueadas, mira el icono que aparece en la barra de direcciones: ahí se permiten para el sitio.

11.2. Documentar objetos de datos con JSDoc

En la UD2 documentaste funciones. Cuando empiezas a manejar objetos (las preferencias, un mensaje de postMessage), conviene documentar su forma con @typedef, y VS Code te autocompletará las propiedades y avisará si escribes mal una:

```
/**
 * Preferencias del usuario que se guardan en el navegador.
 * @typedef {Object} Preferencias
 * @property {number} version      - Versión de la estructura guardada.
 * @property {'claro' | 'oscuro' | 'sistema'} tema
 * @property {string} idioma       - Etiqueta BCP 47, por ejemplo 'es-ES'.
 * @property {string} moneda       - Código ISO 4217, por ejemplo 'EUR'.
 * @property {number} porPagina    - Elementos por página en los listados.
 */

/** @type {Preferencias} */
const PREFERENCIAS_POR_DEFECTO = {
  version: 1,
  tema: 'sistema',
  idioma: navigator.language,
  moneda: 'EUR',
  porPagina: 10,
};
```

Y en el README.md del proyecto, añade un apartado «Datos que se guardan en el navegador» con una tabla: clave, qué contiene, mecanismo, cuánto dura y por qué es necesario. Es documentación para el usuario, para el equipo y para cumplir con el apartado 10.

A10. Preferencias de tu proyecto (producto de la unidad; CE a–h)

Objetivo: construir la primera versión del requisito R6 del proyecto: una página de preferencias que se guarda y se recupera del navegador, con aviso de consentimiento, formato localizado y un reloj en la página de inicio.

Se hace en dwec-proyecto, en una página nueva preferencias.html (con su preferencias.js, almacen.js y la hoja de estilos del proyecto), y se modifica index.html para integrar el reloj y aplicar las preferencias. Las piezas ya las has construido en A3 a A9: aquí las ensamblas.

Requisitos (R = obligatorio, O = opcional):

	Requisito
R1	La página de preferencias muestra cuatro controles: tema (claro / oscuro / sistema), idioma (al menos es-ES, en-GB y otro), moneda (al menos EUR, USD y otra) y elementos por página (5, 10, 20). Puedes usar <select>; para leer su valor basta con document.getElementById('tema').value.
R2	Al pulsar «Guardar», las preferencias se guardan en localStorage como un único objeto JSON con version,

	mediante las funciones guardar y cargar de almacen.js, documentadas con JSDoc y con try…catch. Un @typedef Preferencias documenta la estructura.
R3	Al cargar cualquier página del proyecto, se leen las preferencias guardadas (o las de por defecto) y se aplican: el tema con dataset.tema (si es «sistema», con matchMedia), el atributo lang del <html> y el resto queda disponible para las unidades siguientes.
R4	Una vista previa en la página de preferencias muestra, con las preferencias actuales, un importe de ejemplo (1234.5) formateado en la moneda e idioma elegidos, la fecha de hoy con dateStyle: 'full' y un texto con Intl.RelativeTimeFormat («hace 3 días»). Se actualiza al guardar.
R5	Aviso de consentimiento según el apartado 10.3: aparece solo si no se ha respondido, con «Aceptar» y «Rechazar» igual de visibles y un enlace a un texto que explica qué se guarda. Si se rechaza, las preferencias se aplican pero no se persisten (se mantienen en sessionStorage mientras dure la pestaña).
R6	Un botón «Borrar mis datos» que elimina todo lo guardado por el proyecto (localStorage.clear() y sessionStorage.clear()), vuelve a las preferencias por defecto y muestra de nuevo el aviso. Un mensaje de estado (*toast*, A7) confirma cada acción.
R7	La página de inicio muestra un reloj con la hora actual (HH:MM:SS) y la fecha larga en el idioma de las preferencias, actualizado cada segundo.
R8	El README.md incluye la tabla «Datos que se guardan en el navegador» (apartado 11.2) y una captura de la pestaña Application con las claves del proyecto.
O1	Las preferencias se sincronizan entre pestañas con el evento storage o con BroadcastChannel: al guardar en una, la otra cambia el tema sin recargar.
O2	Un botón «Vista previa en ventana» abre una ventana emergente con la página de inicio y le envía las preferencias con postMessage para que las aplique.
O3	Un selector de zona horaria (Intl.DateTimeFormat admite la opción timeZone) y el reloj muestra la hora de la zona elegida.

Entrega: commit y push en dwec-proyecto con el mensaje UT3: preferencias persistentes (R6). Ten preparado el proyecto para enseñarlo: se te pedirá que expliques una parte del código.

12. Resumen de actividades y entregas

Actividad	Apartado	Sesión orientativa	CE	Entrega
A1. Explorando los objetos predefinidos	1	1	a, h	a1-objetos + NOTAS.md
A2. Laboratorio de cadenas y números	2	1–2	a, h	a2-cadenas + PRUEBAS.md
A3. Fechas e importes bien formateados	3	2	a, c, h	a3-fechas + PRUEBAS.md
A4. Reloj y cuenta atrás	4	3	a, c, h	a4-tiempo
A5. La página habla	5	4	c, d, h	a5-pagina + NOTAS.md
A6. Radiografía del navegador	6	4–5	b, c, h	a6-bom + NOTAS.md
A7. Diálogos propios	7	5	e, h	a7-dialogos + NOTAS.md
A8. Ventanas que se hablan	8	6	b, f, h	a8-ventanas + NOTAS.md
A9. Guardar y recuperar	9	6–7	g, h	a9-almacen + NOTAS.md
A10. Preferencias de tu proyecto	10–11	7–8	a–h	dwec-proyecto (R6)
Prueba práctica	—	8	a–h	En ordenador

13. Autoevaluación

1. Clasifica Math, localStorage, document y JSON en las tres familias del apartado 1. ¿Cuáles existen en Node.js?
2. ¿Qué diferencia hay entre Date.now() y new Date().getTime()? ¿Y entre un método estático y uno de instancia?
3. ¿Qué devuelve '2026-10-13'.slice(-2)? ¿Y ' DAW '.trim().toLowerCase().padStart(5, '*')?
4. Escribe la expresión que da un entero aleatorio entre 10 y 20, ambos incluidos. ¿Por qué no sirve Math.random() para generar contraseñas?
5. ¿Qué fecha representa new Date(2026, 11, 1)? ¿Y por qué new Date('2026-12-01') puede mostrar el 30 de noviembre en algunos lugares del mundo?
6. ¿Qué muestra new Intl.NumberFormat('de-DE', { style: 'currency', currency: 'EUR' }).format(1234.5)? ¿Y con 'es-ES'? ¿Y con 'en-US' y USD?
7. ¿Qué imprime console.log('a'); setTimeout(() => console.log('b'), 0); console.log('c'); y por qué?
8. ¿Cuál es el error en setInterval(actualizar(), 1000)? ¿Cómo se corrige? ¿Qué pasa si nunca llamas a clearInterval?
9. Un compañero inserta el nombre que escribe el usuario con innerHTML. ¿Qué riesgo tiene y cuáles son las dos formas de evitarlo?
10. Para la URL https://tienda.es/catalogo/index.html?cat=libros&pag=2#top, indica location.origin, location.pathname, location.search y location.hash. ¿Cómo obtendrías el valor de pag como número?
11. ¿Qué diferencia hay entre location.assign y location.replace? ¿Y entre confirm y un <dialog> con showModal() respecto a cómo recibes la respuesta?
12. ¿Por qué window.open puede devolver null? ¿Qué debes comprobar siempre en el receptor de un postMessage?
13. Tienes la aplicación abierta en dos pestañas. Nombra dos mecanismos para que un cambio en una se refleje en la otra y di cuál usarías si el dato ya se guarda en localStorage.
14. Completa la tabla: cookie, localStorage, sessionStorage: ¿cuánto dura cada uno? ¿Cuál viaja al servidor? ¿Cuál se comparte entre pestañas?
15. ¿Por qué localStorage.setItem('prefs', { tema: 'oscuro' }) no hace lo que parece? ¿Qué dos funciones lo arreglan y por qué conviene rodearlas con try…catch?
16. ¿Qué información puede guardar una aplicación sin pedir consentimiento? Pon dos ejemplos de cada tipo y explica qué debe ofrecer el aviso para ser válido.

14. Recursos

- MDN · Objetos globales estándar (en español): https://developer.mozilla.org/es/docs/Web/JavaScript/Reference/Global_Objects — referencia de String, Number, Math, Date, JSON e Intl.
- MDN · Intl: https://developer.mozilla.org/es/docs/Web/JavaScript/Reference/Global_Objects/Intl — todos los formateadores con ejemplos.
- MDN · Window: https://developer.mozilla.org/es/docs/Web/API/Window — location, history, navigator, screen, open, postMessage, temporizadores.
- MDN · API de almacenamiento web: https://developer.mozilla.org/es/docs/Web/API/Web_Storage_API — localStorage, sessionStorage y el evento storage.
- MDN · document.cookie: https://developer.mozilla.org/es/docs/Web/API/Document/cookie

- MDN · Uso de IndexedDB: https://developer.mozilla.org/es/docs/Web/API/IndexedDB_API/Using_IndexedDB
- MDN · El elemento <dialog>: https://developer.mozilla.org/es/docs/Web/HTML/Element/dialog
- javascript.info · Fechas y hora (en español): https://es.javascript.info/date · Cadenas: https://es.javascript.info/string · Números: https://es.javascript.info/number
- javascript.info · localStorage y sessionStorage: https://es.javascript.info/localstorage · Cookies: https://es.javascript.info/cookie
- javascript.info · Ventanas emergentes y comunicación entre ventanas: https://es.javascript.info/popup-windows y https://es.javascript.info/cross-window-communication
- javascript.info · setTimeout y setInterval: https://es.javascript.info/settimeout-setinterval
- Chrome DevTools · Ver y editar el almacenamiento: https://developer.chrome.com/docs/devtools/storage/localstorage
- AEPD · Guía sobre el uso de las cookies: https://www.aepd.es/guias/guia-cookies.pdf
- web.dev · Entender las cookies: https://web.dev/articles/understanding-cookies
- Manual de apoyo (Ed. Síntesis): capítulo 3, apartados 3.2 (tiempo), 3.3 (cookies) y 3.4 (almacenamiento local); apartado 4.10 (BOM).